

Assignment # 3



Submitted by:

Name	Registration No.
Muhammad Abdullah	FA24-BSE-042

Submitted to: Mam Rubina

Course: DBMS

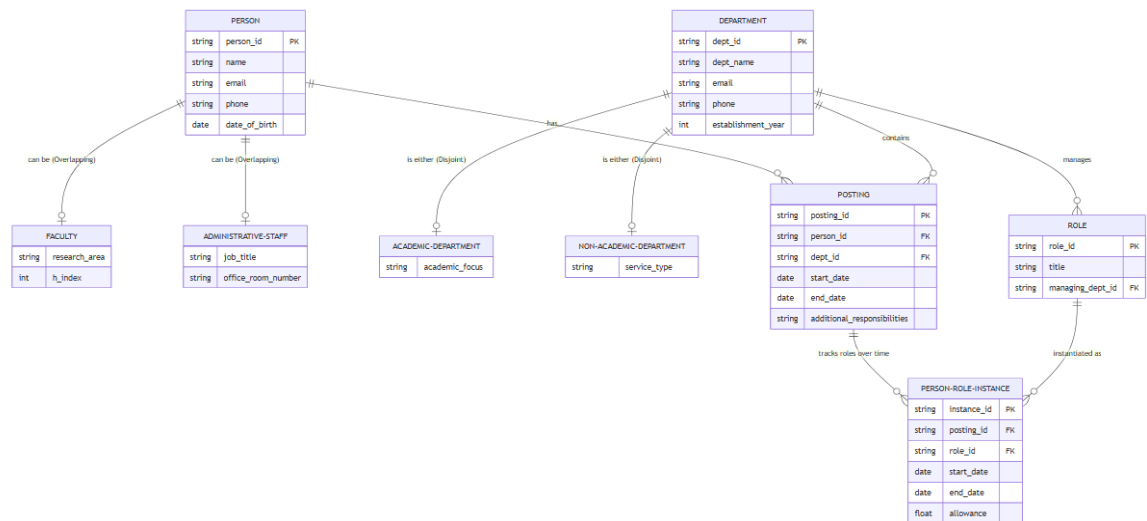
Dated: May 21, 2026

**DEPARTMENT OF COMPUTER SCIENCE
COMSATS UNIVERSITY ISLAMABAD,
ISLAMABAD CAMPUS**

Problem 1: COMSATS University Islamabad Academic System [2 Marks]

This problem involves modeling the COMSATS academic structure, including persons (faculty and administrative staff), departments, postings, and roles using an Enhanced Entity-Relationship (EER) diagram and relational schema up to 3NF.

1.1 EER Diagram Description



1.2 Relational Schema

After transforming the EER diagram:

Person(`person_id` PK, `name`, `email`, `phone`, `date_of_birth`)

Faculty(`person_id` FK->Person PK, `research_area`, `h_index`)

AdministrativeStaff(`person_id` FK->Person PK, `job_title`, `office_room_number`)

Department(`dept_id` PK, `dept_name`, `email`, `phone`, `establishment_year`)

AcademicDepartment(`dept_id` FK->Department PK, `academic_focus`)

NonAcademicDepartment(`dept_id` FK->Department PK, `service_type`)

Posting(`posting_id` PK, `person_id` FK->Person, `dept_id` FK->Department, `start_date`, `end_date`, `additional_responsibilities`)

Role(`role_id` PK, `title`, `managing_dept_id` FK->Department)

PersonRoleInstance(`instance_id` PK, `posting_id` FK->Posting, `role_id` FK->Role, `start_date`, `end_date`, `allowance`)

1.3 Normalization to 3NF

First Normal Form (1NF)

All attributes are atomic (no multi-valued or composite attributes stored directly). All tables have a primary key. Achieved.

Second Normal Form (2NF)

No partial dependencies exist because all non-composite primary keys are used. In Posting, both person_id and dept_id together are meaningful, but posting_id is surrogate PK. PersonRoleInstance uses instance_id as surrogate PK. All non-key attributes depend fully on the full PK. Achieved.

Third Normal Form (3NF)

No transitive dependencies:

- In Faculty, research_area and h_index depend only on person_id (PK), not on each other transitively.
- In Posting, start_date, end_date, and responsibilities all depend directly on posting_id.
- In PersonRoleInstance, allowance depends directly on instance_id (the specific occurrence of a role), not transitively on role_id. If allowance were determined only by the role, it would move to the Role table.
- All tables satisfy 3NF.

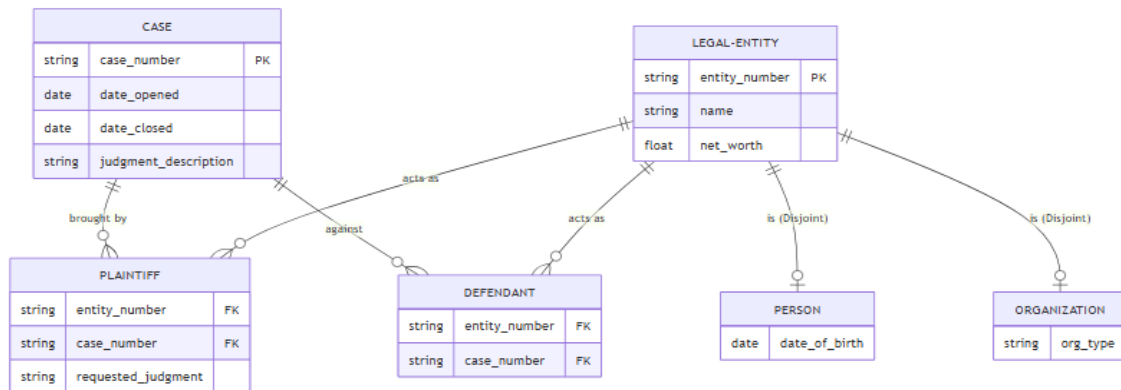
1.4 Final 3NF Schema Tables

Table Name	Primary Key	Foreign Keys	Other Attributes
Person	person_id	—	name, email, phone, dob
Faculty	person_id	person_id -> Person	research_area, h_index
AdministrativeStaff	person_id	person_id -> Person	job_title, office_room_no
Department	dept_id	—	dept_name, email, phone, estab_year
AcademicDepartment	dept_id	dept_id -> Department	academic_focus
NonAcademicDepartment	dept_id	dept_id -> Department	service_type
Posting	posting_id	person_id->Person, dept_id->Dept	start_date, end_date, responsibilities
Role	role_id	managing_dept_id -> Department	title
PersonRoleInstance	instance_id	posting_id->Posting, role_id->Role	start_date, end_date, allowance

Problem 2: Database for Law Firm [2 Marks]

This problem models a law firm's case management system, including cases, plaintiffs, defendants, and legal entities (persons or organizations).

2.1 EER Diagram Description



2.2 Relational Schema

LegalEntity(entity_number PK, name, net_worth, entity_type CHECK('Person','Organization'))

Person(entity_number FK->LegalEntity PK, date_of_birth)

Organization(entity_number FK->LegalEntity PK, org_type)

Case(case_number PK, date_opened, date_closed, judgment_description)

Plaintiff(entity_number FK->LegalEntity, case_number FK->Case, requested_judgment, PRIMARY KEY(entity_number, case_number))

Defendant(entity_number FK->LegalEntity, case_number FK->Case, PRIMARY KEY(entity_number, case_number))

2.3 Normalization to 3NF

1NF

All attributes are atomic; every table has a defined primary key. Achieved.

2NF

Plaintiff table has composite PK (entity_number, case_number). The attribute requested_judgment is a characteristic of the plaintiff's role in that specific case — it depends on the full composite key, not a partial key. Achieved.

3NF

No transitive functional dependencies:

- In Case: date_opened, date_closed, judgment_description all depend directly on case_number.

- In LegalEntity: name and net_worth depend on entity_number; entity_type is stored but no attribute depends on entity_type transitively.
- In Plaintiff: requested_judgment depends on (entity_number, case_number) — no transitivity.
- All tables are in 3NF.

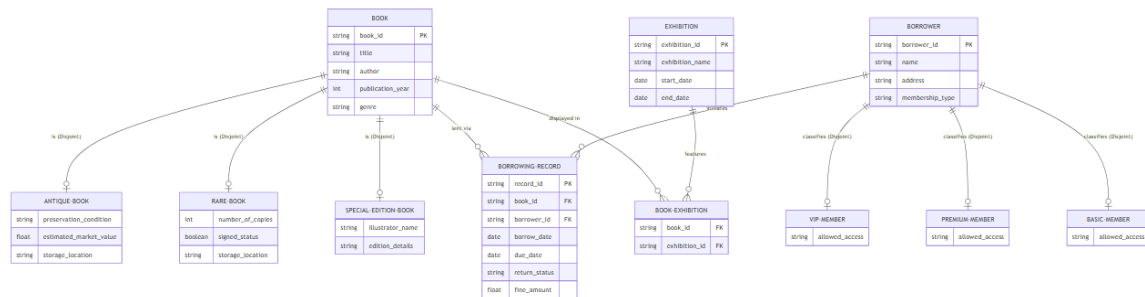
2.4 Final 3NF Schema Tables

Table Name	Primary Key	Foreign Keys	Other Attributes
LegalEntity	entity_number	—	name, net_worth, entity_type
Person	entity_number	entity_number -> LegalEntity	date_of_birth
Organization	entity_number	entity_number -> LegalEntity	org_type
Case	case_number	—	date_opened, date_closed, judgment_description
Plaintiff	(entity_number, case_number)	entity_number->LegalEntity, case_number->Case	requested_judgment
Defendant	(entity_number, case_number)	entity_number->LegalEntity, case_number->Case	—

Problem 3: Library Management System for Rare Books [2 Marks]

This problem models a specialized library that manages rare, antique, and special edition books along with borrowers of different membership tiers and exhibitions.

3.1 EER Diagram Description



3.2 Relational Schema

Book(`book_id` PK, `title`, `author`, `publication_year`, `genre`, `book_type` CHECK('Antique','Rare','SpecialEdition'))

AntiqueBook(`book_id` FK->`Book` PK, `preservation_condition`, `estimated_market_value`)

RareBook(`book_id` FK->`Book` PK, `number_of_copies`, `signed_status`)

SpecialEditionBook(`book_id` FK->`Book` PK, `illustrator_name`, `edition_details`)

Borrower(`borrower_id` PK, `name`, `address`, `membership_type` CHECK('Basic','Premium','VIP'))

BorrowingRecord(`record_id` PK, `book_id` FK->`Book`, `borrower_id` FK->`Borrower`, `borrow_date`, `due_date`, `return_status`, `fine_amount`)

Exhibition(`exhibition_id` PK, `exhibition_name`, `start_date`, `end_date`)

BookExhibition(`book_id` FK->`Book`, `exhibition_id` FK->`Exhibition`, PRIMARY KEY(`book_id`, `exhibition_id`))

3.3 Normalization to 3NF

1NF

All fields atomic. PKs defined. `fine_amount` is stored as NULL when `return_status` = 'Returned'. Achieved.

2NF

BorrowingRecord uses surrogate PK `record_id`, so no partial dependency. **BookExhibition** has composite PK with no non-key attributes. Achieved.

3NF

Checking for transitive dependencies:

- In BorrowingRecord: fine_amount depends on return_status ('Overdue') — this is a conditional attribute but not a transitive dependency since fine_amount is determined by the specific record (record_id), not via a functional chain through another non-key attribute. If fine policy were standardized per membership tier, we might need a separate FinePolicy table, but as stated fine_amount is per record.
- In Borrower: membership_type does not determine name or address — no transitivity. If we needed to store membership-tier features separately, we could, but that is not required here.
- All tables satisfy 3NF.

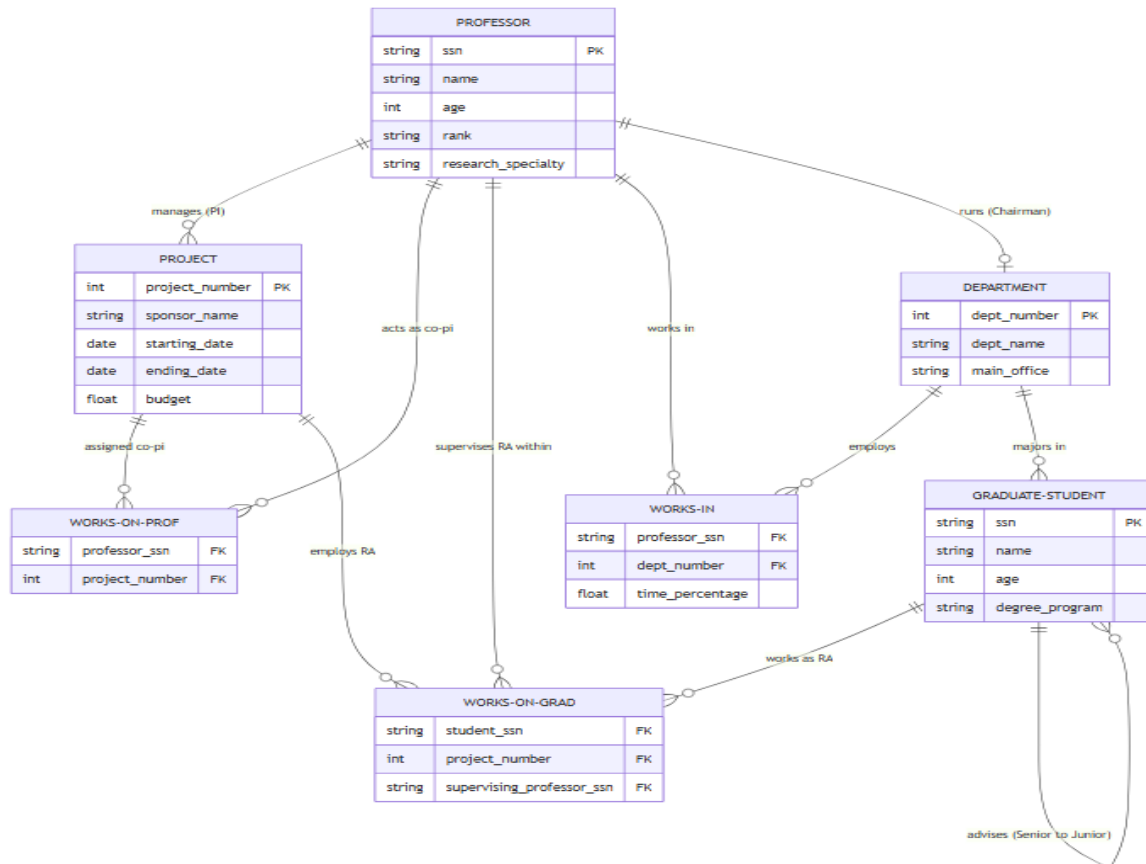
3.4 Final 3NF Schema Tables

Table Name	Primary Key	Foreign Keys	Other Attributes
Book	book_id	—	title, author, pub_year, genre, book_type
AntiqueBook	book_id	book_id -> Book	preservation_condition, market_value
RareBook	book_id	book_id -> Book	num_copies, signed_status
SpecialEditionBook	book_id	book_id -> Book	illustrator_name, edition_details
Borrower	borrower_id	—	name, address, membership_type
BorrowingRecord	record_id	book_id->Book, borrower_id->Borrower	borrow_date, due_date, return_status, fine_amount
Exhibition	exhibition_id	—	exhibition_name, start_date, end_date
BookExhibition	(book_id, exhibition_id)	book_id->Book, exhibition_id->Exhibition	—

Problem 4: Faculty Database [2 Marks]

This problem models a university research environment including professors, graduate students, projects, departments, and various supervisory/advisory relationships.

4.1 EER Diagram Description



4.2 Relational Schema

Professor(ssn PK, name, age, rank, research_specialty)

GraduateStudent(ssn PK, name, age, degree_program, major_dept_number FK->Department, advisor_ssn FK->GraduateStudent)

Project(project_number PK, sponsor_name, starting_date, ending_date, budget, pi_ssn FK->Professor)

Department(dept_number PK, dept_name, main_office, chairman_ssn FK->Professor)

ProfProject(prof_ssn FK->Professor, project_number FK->Project, PRIMARY KEY(prof_ssn, project_number))

GradProject(student_ssn FK->GraduateStudent, project_number FK->Project, supervisor_ssn FK->Professor, PRIMARY KEY(student_ssn, project_number))

ProfDept(prof_ssn FK->Professor, dept_number FK->Department, time_percentage, PRIMARY KEY(prof_ssn, dept_number))

4.3 Normalization to 3NF

1NF

All attributes atomic. All tables have defined primary keys. Achieved.

2NF

Checking composite PKs for partial dependencies:

- ProfProject: PK is (prof_ssn, project_number) — no non-key attributes, so no partial dependency.
- GradProject: PK is (student_ssn, project_number). The attribute supervisor_ssn depends on the full composite key (which professor supervises a specific student on a specific project), not on student alone or project alone. Achieved.
- ProfDept: PK is (prof_ssn, dept_number). Attribute time_percentage depends on the combination. Achieved.

3NF

Checking transitive dependencies:

- In Professor: rank, research_specialty, name, age all depend directly on ssn. No transitivity.
- In GraduateStudent: major_dept_number and advisor_ssn are FKs — they represent relationships, not transitive FDs.
- In Project: pi_ssn (FK to Professor) — no attribute of Professor is stored here (no transitivity). Budget, dates, sponsor all depend on project_number.
- In Department: chairman_ssn (FK to Professor) — no transitivity.
- All tables are in 3NF.

4.4 Final 3NF Schema Tables

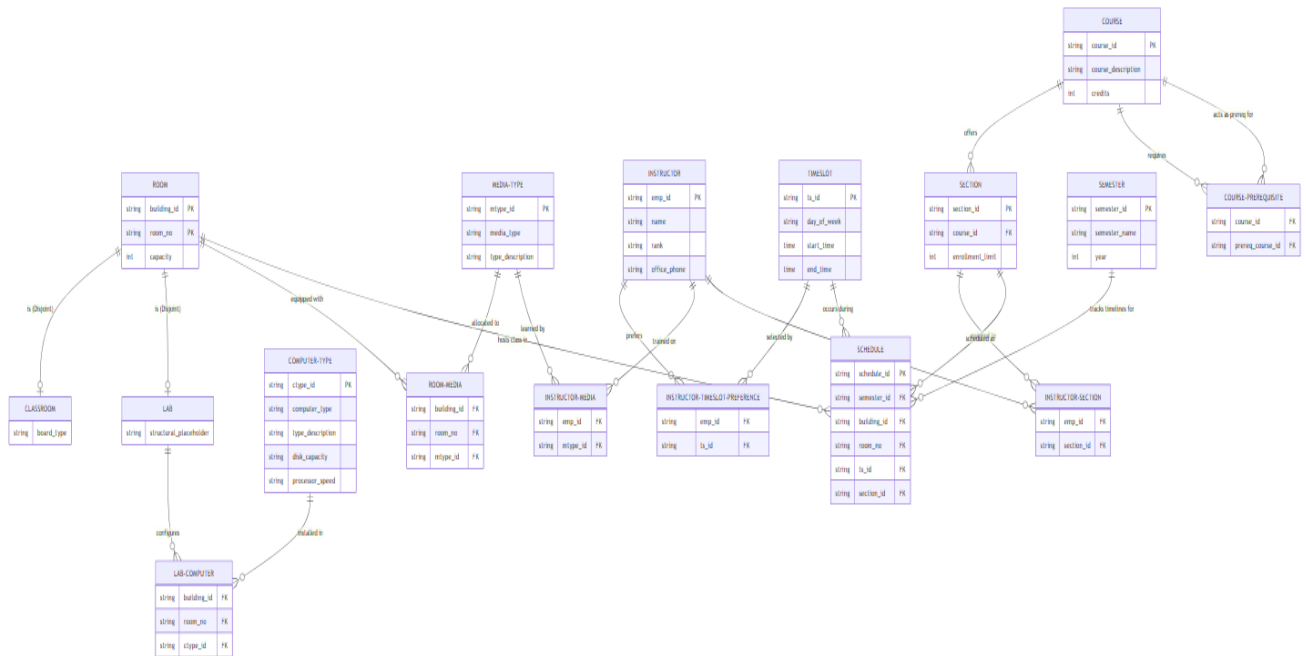
Table Name	Primary Key	Foreign Keys	Other Attributes
Professor	ssn	—	name, age, rank, research_specialty
GraduateStudent	ssn	major_dept_number->Dept, advisor_ssn->GradStudent	name, age, degree_program
Project	project_number	pi_ssn -> Professor	sponsor_name, start_date, end_date, budget
Department	dept_number	chairman_ssn -> Professor	dept_name, main_office

ProfProject	(prof_ssn, project_number)	prof_ssn->Prof, project_number->Project	—
GradProject	(student_ssn, project_number)	student_ssn->Grad, project_number->Project, supervisor_ssn->Professor	—
ProfDept	(prof_ssn, dept_number)	prof_ssn->Prof, dept_number->Dept	time_percentage

Problem 5: Scheduling Classes Database [2 Marks]

This problem models a class scheduling system for an international school of technology, covering rooms, media types, computer types, instructors, courses, sections, timeslots, and semester schedules.

5.1 EER Diagram Description



5.2 Relational Schema

Room(building_id, room_no, capacity, room_type CHECK('Lab','Classroom'), PRIMARY KEY(building_id, room_no))

Classroom(building_id, room_no, board_type, PRIMARY KEY(building_id, room_no), FK->(building_id, room_no)->Room)

Lab(building_id, room_no, PRIMARY KEY(building_id, room_no), FK->(building_id, room_no)->Room)

MediaType(mtype_id PK, media_type, type_description)

ComputerType(ctype_id PK, computer_type, type_description, disk_capacity, processor_speed)

Instructor(emp_id PK, name, rank, office_phone)

Timeslot(ts_id PK, day_of_week, start_time, end_time)

Course(course_id PK, course_description, credits)

CoursePrerequisite(course_id FK->Course, prereq_id FK->Course, PRIMARY KEY(course_id, prereq_id))

Section(section_id PK, course_id FK->Course, enrollment_limit)

Semester(semester_id PK, semester_name, year)

Schedule(schedule_id PK, semester_id FK->Semester, building_id, room_no, ts_id FK->Timeslot, section_id FK->Section, FK(building_id, room_no)->Room, UNIQUE(semester_id, building_id, room_no, ts_id))

InstructorSection(emp_id FK->Instructor, section_id FK->Section, PRIMARY KEY(emp_id, section_id))

InstructorTimeslotPref(emp_id FK->Instructor, ts_id FK->Timeslot, PRIMARY KEY(emp_id, ts_id))

RoomMedia(building_id, room_no, mtype_id FK->MediaType, PRIMARY KEY(building_id, room_no, mtype_id), FK(building_id, room_no)->Room)

InstructorMedia(emp_id FK->Instructor, mtype_id FK->MediaType, PRIMARY KEY(emp_id, mtype_id))

LabComputer(building_id, room_no, ctype_id FK->ComputerType, PRIMARY KEY(building_id, room_no, ctype_id), FK(building_id, room_no)->Lab)

5.3 Normalization to 3NF

1NF

All attributes are atomic. Room's composite PK (building_id, room_no) is valid. All M:N relationships resolved into associative tables. Achieved.

2NF

Checking all composite PKs:

- Schedule: schedule_id is surrogate PK — no partial dependencies on composite FKs.
- CoursePrerequisite, InstructorSection, etc.: no non-key attributes, so trivially in 2NF.
- RoomMedia, LabComputer: no non-key attributes.
- Achieved.

3NF

Checking transitive dependencies:

- In Room: capacity depends on (building_id, room_no). room_type does not cause any attribute to depend on it transitively (Lab/Classroom subtype data is in separate tables).
- In Classroom: board_type depends on the room PK — no transitivity.
- In ComputerType: all attributes depend on ctype_id. processor_speed and disk_capacity describe the type, not anything else.
- In Schedule: section_id FK points to Section — we do not replicate Section attributes here (no transitivity).
- All tables are in 3NF.

5.4 Final 3NF Schema Tables

Table Name	Primary Key	Foreign Keys	Other Attributes
Room	(building_id, room_no)	—	capacity, room_type

Classroom	(building_id, room_no)	(building_id, room_no) -> Room	board_type
Lab	(building_id, room_no)	(building_id, room_no) -> Room	—
MediaType	mtype_id	—	media_type, type_description
ComputerType	ctype_id	—	computer_type, type_desc, disk_cap, proc_speed
Instructor	emp_id	—	name, rank, office_phone
Timeslot	ts_id	—	day_of_week, start_time, end_time
Course	course_id	—	course_description, credits
CoursePrerequisite	(course_id, prereq_id)	course_id->Course, prereq_id->Course	—
Section	section_id	course_id -> Course	enrollment_limit
Semester	semester_id	—	semester_name, year
Schedule	schedule_id	semester_id, (bldg,room)->Room, ts_id, section_id	UNIQUE(sem, room, ts)
InstructorSection	(emp_id, section_id)	emp_id->Instructor, section_id->Section	—
InstructorTimeslotPref	(emp_id, ts_id)	emp_id->Instructor, ts_id->Timeslot	—
RoomMedia	(bldg, room_no, mtype_id)	(bldg,room)->Room, mtype_id->MediaType	—
InstructorMedia	(emp_id, mtype_id)	emp_id->Instructor, mtype_id->MediaType	—
LabComputer	(bldg, room_no, ctype_id)	(bldg,room)->Lab, ctype_id->ComputerType	—

